

deeplearning.ai

Basics of Neural Network Programming

Vectorization

What is vectorization?

Getting rid of for loops in your code.

Logistic requires computation of

$z = w^T x + b$, where $w \in \mathbb{R}^{n_x}$, $x \in \mathbb{R}^{n_x}$, $w = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}$, $x = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}$

Non-vectorized implementation

```
z=0
for i in range(n_x):
    z += w[i]*x[i]
z += b
```

Vectorized Implementation

```
z = np.dot(w,x) + b
```

($w^T x$) → much faster

Makes use of parallelization:

GPU } have parallelization instructions

CPU } SIMD - single instruction multiple data

Andrew Ng

Question

True or false. Vectorization cannot be done without a GPU.

- ○ False
- ○ True



deeplearning.ai

Basics of Neural Network Programming

More vectorization examples

Neural network programming guideline

Whenever possible, avoid explicit for-loops.

Neural network programming guideline

Whenever possible, avoid explicit for-loops.

$u = Av$ (matrix, vector)

$u_i = \sum_j A_{ij} v_j$

implementation (non-vectorized)

```
u = np.zeros((n,1))
for i...
    for j...
        u[i] += A[i][j] * v[j]
```

vectorized implementation

```
u = np.dot(A,v)
```

→ much faster

Andrew Ng

Vectors and matrix valued functions

Say you need to apply the exponential operation on every element of a matrix/vector.

$$v = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \rightarrow u = \begin{bmatrix} e^{v_1} \\ e^{v_2} \\ \vdots \\ e^{v_n} \end{bmatrix}$$

(non-vectorized)

```
u = np.zeros((n,1))
for i in range(n):
    u[i]=math.exp(v[i])
```

Vectorized implementation

```
import numpy as np
u=np.exp(v)
```

→ elementwise exponential

other functions of numpy

```
np.log(v)
np.abs(v)
np.maximum(v,0)
v**2
1/v
```

→ elementwise log

→ elementwise max

→ elementwise square

Logistic regression derivatives

$J = 0,\ dw_1 = 0,\ dw_2 = 0,\ db = 0$

for i = 1 to m:

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -\left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right]$

$dz^{(i)} = a^{(i)}(1 - a^{(i)})$

for j=1...n_x:

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

$dw_j \mathrel{+}= \ldots$

$db \mathrel{+}= dz^{(i)}$

$n_x = 2$, want to eliminate the second for loop

$J = J/m,\ dw_1 = dw_1/m,\ dw_2 = dw_2/m,\ db = db/m$

Andrew Ng

Logistic regression derivatives

$J = 0$, $dw_1 = 0$, $dw_2 = 0$, $db = 0$

Replace with: $dw = np.zeros((n_x, 1))$

for $i = 1$ to m:

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -\left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right]$

$dz^{(i)} = a^{(i)}(1 - a^{(i)})$

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

Replace with: $dw \mathrel{+}= x^{(i)} dz^{(i)}$, $dw = \begin{bmatrix} dw_1 \\ dw_2 \end{bmatrix}$

$db \mathrel{+}= dz^{(i)}$

$J = J/m$, $dw_1 = dw_1/m$, $dw_2 = dw_2/m$, $db = db/m$

$dw /= m$

We still have the loop that goes over each example.

Andrew Ng



deeplearning.ai

Basics of Neural Network Programming

Vectorizing Logistic Regression

Goal: To implement Logistic Regression without using any explicit for loop.

Vectorizing Logistic Regression

$z^{(1)} = w^T x^{(1)} + b$

$a^{(1)} = \sigma(z^{(1)})$

prediction for the 1st example

$z^{(2)} = w^T x^{(2)} + b$

$a^{(2)} = \sigma(z^{(2)})$

prediction for the 2nd example

$z^{(3)} = w^T x^{(3)} + b$

$a^{(3)} = \sigma(z^{(3)})$

prediction for the 3rd example

- To implement the forward propagation (i.e. to compute predictions $a^{(1)}, a^{(2)}, a^{(3)}$...) there is a way without using any for loop.

Andrew Ng

- Define matrix X to contain the training examples in its columns:

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{(n_x \times m)}$$

(n_x, m)

- Compute $z^{(1)}, z^{(2)} \ldots$ in one step. Define the vector Z:

$$Z = \begin{bmatrix} z^{(1)} & z^{(2)} & \cdots & z^{(m)} \end{bmatrix} = w^T X + \begin{bmatrix} b & b & \cdots & b \end{bmatrix}_{1 \times m}$$

$$\begin{bmatrix} — \end{bmatrix} \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$= \begin{bmatrix} w^T x^{(1)} & w^T x^{(2)} & \cdots & w^T x^{(m)} \end{bmatrix} + \begin{bmatrix} b & b & \cdots & b \end{bmatrix}$$

$$Z = \begin{bmatrix} z^{(1)} & z^{(2)} & \cdots & z^{(m)} \end{bmatrix} = \begin{bmatrix} \underbrace{w^T x^{(1)} + b}_{z^{(1)}} & \underbrace{w^T x^{(2)} + b}_{z^{(2)}} & \cdots & \underbrace{w^T x^{(m)} + b}_{z^{(m)}} \end{bmatrix}_{1 \times m}$$

in Python:

```
Z = np.dot(w.T, X) + b
```

↳ $\in \mathbb{R}, (1,1)$

Broadcasting: number b is "turned" into a $1 \times m$ vector before this addition.

$$A = \begin{bmatrix} a^{(1)} & a^{(2)} & \cdots & a^{(m)} \end{bmatrix} = \sigma(Z)$$

↳ vector

↳ vector-valued sigmoid function.



deeplearning.ai

Basics of Neural Network Programming

Vectorizing Logistic Regression's Gradient Computation

Vectorizing Logistic Regression

$dz^{(1)} = a^{(1)} - y^{(1)}$ $dz^{(2)} = a^{(2)} - y^{(2)}$ ---

Define the vector:

$$dZ = [dz^{(1)} \; dz^{(2)} \; \cdots \; dz^{(m)}]_{1 \times m}$$

$$A = [a^{(1)} \; a^{(2)} \; \cdots \; a^{(m)}]$$

$$Y = [y^{(1)} \; y^{(2)} \; \cdots \; y^{(m)}]$$

$$dZ = A - Y = [a^{(1)} - y^{(1)} \; \underbrace{a^{(2)} - y^{(2)}}_{2^{nd}\text{ elem.}} \; \cdots]$$

$\underbrace{\quad}_{1^{st}\text{ element}}$

- Previously we eliminated one (inner) loop in gradient descent implementation.

Andrew Ng

- But we still have the loop over the examples which computes:

$dw = 0$
$dw \mathrel{+}= x^{(1)} dz^{(1)}$
$dw \mathrel{+}= x^{(2)} dz^{(2)}$
$\vdots$
$dw \mathrel{/}= m$

$db = 0$
$db \mathrel{+}= dz^{(1)}$
$db \mathrel{+}= dz^{(2)}$
$\vdots$
$db \mathrel{+}= dz^{(m)}$
$db \mathrel{/}= m$

looping over all examples

- The remove the loop over all examples:

$$db = \frac{1}{m} \sum_{i=1}^{m} dz^{(i)} = \frac{1}{m} np.sum(dz)$$

vectorized: no for loop over the examples.

$$dw = \frac{1}{m} X dz^{T}$$

$$= \frac{1}{m} \begin{bmatrix} | & | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & & x^{(m)} \\ | & | & | & & | \end{bmatrix} \begin{bmatrix} dz^{(1)} \\ \vdots \\ dz^{(m)} \end{bmatrix}$$

$$dw = \frac{1}{m} \left[x^{(1)} dz^{(1)} + \cdots + x^{(m)} dz^{(m)} \right]$$

Implementing Logistic Regression

Non-vectorized implementation

$J = 0,\ dw_1 = 0,\ dw_2 = 0,\ db = 0$

for i = 1 to m:

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -\left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})\right]$

$dz^{(i)} = a^{(i)} - y^{(i)}$

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

$db \mathrel{+}= dz^{(i)}$

$J = J/m,\ dw_1 = dw_1/m,\ dw_2 = dw_2/m$

$db = db/m$

Andrew Ng

Implementing Logistic Regression

Non-vectorized implementation

$J = 0, dw_1 = 0, dw_2 = 0, db = 0$

for i = 1 to m:

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -\left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})\right]$

$dz^{(i)} = a^{(i)} - y^{(i)}$

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

$db \mathrel{+}= dz^{(i)}$

Got rid of this for loop

Replaced with: $dw \mathrel{+}= x^{(i)} * dz^{(i)}$

where $dw = [dw_1 \quad dw_2]$

$J = J/m, dw_1 = dw_1/m, dw_2 = dw_2/m$

$db = db/m$

Andrew Ng

Implementing Logistic Regression

needed for multiple iterations of gradient descent

Non-vectorized implementation

$J = 0,\ dw_1 = 0,\ dw_2 = 0,\ db = 0$

for i = 1 to m: → Get rid of this loop

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -\left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})\right]$

$dz^{(i)} = a^{(i)} - y^{(i)}$

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

$db \mathrel{+}= dz^{(i)}$

$J = J/m,\ dw_1 = dw_1/m,\ dw_2 = dw_2/m$

$db = db/m$

for iter in range(1000):

$Z = w^T X + b$

$= np.dot(w.T, x) + b$

$A = \sigma(Z)$ (all a's are computed)

$dZ = A - Y$

$dW = \frac{1}{m} X dZ^T$

$db = \frac{1}{m} np.sum(dZ)$

(forward and back propagation has been done)

$w = w - \alpha dW$

$b = b - \alpha db$

} update parameters

Andrew Ng



deeplearning.ai

Basics of Neural Network Programming

Broadcasting in Python

Broadcasting example

Calories from Carbs, Proteins, Fats in 100g of different foods:

	Apples	Beef	Eggs	Potatoes
Carb	56.0	0.0	4.4	68.0
Protein	1.2	104.0	52.0	8.0
Fat	1.8	135.0	99.0	0.9

Sum = 59

Carb % = $\frac{56}{59} \cdot 100 = 94.9\%$

Goal: Calculate the percentage of Carb, Protein, Fat for each item.
Can you do this without using an explicit for-loop?

Broadcasting example

Calories from Carbs, Proteins, Fats in 100g of different foods:

	Apples	Beef	Eggs	Potatoes
Carb	56.0	0.0	4.4	68.0
Protein	1.2	104.0	52.0	8.0
Fat	1.8	135.0	99.0	0.9

$= A_{(3,4)}$

```
cal = A.sum(axis = 0) #sum over the columns ↓0 1→
percentage = 100*A/(cal.reshape(1,4))
```

broadcasting (3,4)/(1,4)

how can you do this division?

HW: Try the code!

cal is already (1,4) not needed

Broadcasting example

auto-expand the number to the same size

- Addition of a vector and number:

$$\begin{bmatrix}1\\2\\3\\4\end{bmatrix} + 100 \longrightarrow \begin{bmatrix}1\\2\\3\\4\end{bmatrix} + \begin{bmatrix}100\\100\\100\\100\end{bmatrix} = \begin{bmatrix}101\\102\\103\\104\end{bmatrix}$$

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100 & 200 & 300\end{bmatrix}$$

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100\\200\end{bmatrix}$$

Broadcasting example

auto-expand the number to the same size

- Addition of a vector and number:

$$\begin{bmatrix}1\\2\\3\\4\end{bmatrix} + 100 \longrightarrow \begin{bmatrix}1\\2\\3\\4\end{bmatrix} + \begin{bmatrix}100\\100\\100\\100\end{bmatrix} = \begin{bmatrix}101\\102\\103\\104\end{bmatrix}$$

- Add a (m,n) matrix with a (1,n) vector

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100 & 200 & 300\end{bmatrix}$$

(m,n) (1,n)

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100\\200\end{bmatrix}$$

Broadcasting example

auto-expand the number to the same size

- Addition of a vector and number:

$$\begin{bmatrix}1\\2\\3\\4\end{bmatrix} + 100 \longrightarrow \begin{bmatrix}1\\2\\3\\4\end{bmatrix} + \begin{bmatrix}100\\100\\100\\100\end{bmatrix} = \begin{bmatrix}101\\102\\103\\104\end{bmatrix}$$

- Add a (m,n) matrix with a (1,n) vector

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100 & 200 & 300\\100 & 200 & 300\end{bmatrix} = \begin{bmatrix}101 & 102 & 103\\104 & 105 & 106\end{bmatrix}$$

(m,n) — copy the row — (1,n) → (m,n)

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100\\200\end{bmatrix}$$

Broadcasting example

$$\begin{bmatrix}1\\2\\3\\4\end{bmatrix} + 100$$

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix} + \begin{bmatrix}100 & 200 & 300\end{bmatrix}$$

- Add

$$\underset{(m,n)}{\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix}} + \underset{(m,1)}{\begin{bmatrix}100\\200\end{bmatrix}} \xrightarrow{\text{copy } n \text{ times}} \underset{(m,n)}{\begin{bmatrix}100 & 100 & 100\\200 & 200 & 200\end{bmatrix}} = \begin{bmatrix}101 & 102 & 103\\204 & 205 & 206\end{bmatrix}$$

General Principle

(m,n) matrix $\pm$ $/$ $*$

(1,n) $\xrightarrow{\text{copy}}$ (m,n) → apply operation

(m,1) $\xrightarrow{\text{copy}}$ (m,n) → element-wise

(m,1) + $\mathbb{R}$

$\begin{bmatrix}1\\2\\3\end{bmatrix}$ + 100 = $\begin{bmatrix}101\\102\\103\end{bmatrix}$

[1 2 3] + 100 = [101 102 103]

- These are the main forms of broadcasting.

Question

Which of the following numpy line of code would sum the values in a matrix A vertically?

- ◯ A.sum(axis = 0)
- ◯ A.sum(axis = 1)
- ◯ A.sum(axis)

A note on python/numpy vectors

- Tips to reduce bugs.
- Some unintuitive results may happen when working with vectors and matrices in Python:

eg.

```
In [1]: import numpy as np

        a = np.random.randn(5)  # generates 5 random Gaussian numbers

In [2]: print(a)
        [ 0.50290632 -0.29691149  0.95429604 -0.82126861 -1.46269164]

In [3]: print(a.shape)
        (5,)  # (5,) structure. Rank 1 array. Neither a row vector
              nor a column vector. May lead to unintuitive effects.

In [4]: print(a.T)
        [ 0.50290632 -0.29691149  0.95429604 -0.82126861 -1.46269164]  → same as a!

In [5]: print(np.dot(a,a.T))  # maybe outer product, a matrix?
        4.06570109321  ------> # but we get a number
```

```
In [6]: a = np.random.randn(5,1)
        print(a)
        [[-0.0967311 ]
         [-2.38617377]
         [-0.3243588 ]
         [-0.96216349]
         [ 0.54410384]]
```

→ 5×1 column vector

```
In [7]: print(a.T)
        [[-0.0967311  -2.38617377 -0.3243588  -0.96216349  0.54410384]]  # row vector

In [8]: print(np.dot(a,a.T))  # outer product
        [[ 0.00935691  0.23081721  0.03137558  0.09307113 -0.05263176]
         [ 0.23081721  5.69382526  0.77397645  2.29588928 -1.2983263 ]
         [ 0.03137558  0.77397645  0.10520863  0.31208619 -0.17648487]
         [ 0.09307113  2.29588928  0.31208619  0.92575858 -0.52351684]
         [-0.05263176 -1.2983263  -0.17648487 -0.52351684  0.29604898]]
```

a^T is a row vector

2 brackets: this is a 1×5 matrix.

Python / Numpy Vectors

```
a = np.random.randn(5)
a.shape = (5,)
```

↝ "rank 1 array"

} Don't use

it doesn't behave consistently as either a row vector or column vector, which makes some of its effects unintuitive.

⇒ Suggestion: don't use rank-1 arrays.

Instead use:

```
a = np.random.randn(5,1) # column vector, 5x1 matrix
a.shape = (5,1)

a = np.random.randn(1,5) # row vector
a.shape = (1,5)
```

↝ matrix of size (1,5)

```
assert(a.shape == (5,1))
```

↝ use this to make sure that a is of size 5x1. (throw assertions).

You can reshape a rank-1 array:

```
a = a.reshape((5,1))
```

(to make it behave more consistently)

Summary: eliminate rank-1 arrays from your code

Question

What kind of array has dimensions in this format: (10,) ?

- ◯ A rank 1 array
- ◯ An identity array
- ◯ A rank 0 array

EXPLANATION OF LOGISTIC REGRESSION COST FUNCTION

$\hat{y} = \sigma(w^T x + b)$ where $\sigma(z) = \frac{1}{1+e^{-z}}$

Interpret $\hat{y} = p(y=1|x)$

If $y=1$: $p(y|x) = \hat{y}$

If $y=0$: $p(y|x) = 1-\hat{y}$

} $p(y|x)$

Summarize the above two equations into a single equation:

$$p(y|x) = \hat{y}^y (1-\hat{y})^{(1-y)}$$

if $y=1$: $p(y|x) = \hat{y} \cdot \underbrace{(1-\hat{y})^0}_{=1} = \hat{y}$

if $y=0$: $p(y|x) = 1 \cdot (1-\hat{y}) = 1-\hat{y}$

Since log function is monotonically increasing:

maximizing $p(y|x)$ is equivalet to maximizing

$$\log p(y|x) = \log \hat{y}^y (1-\hat{y})^{(1-y)} = y \log \hat{y} + (1-y) \log (1-\hat{y})$$

$$= -\mathcal{L}(\hat{y}, y)$$

↳ loss function on a single example

maximizing the probability is equivalet to minimizing the loss function.

Cost on m examples

$$p(\text{labels in training set}) = \prod_{i=1}^{m} p(y^{(i)} \mid x^{(i)})$$ (if examples are IID)

↳ we want to find the parameters that maximize this (i.e. want to maximize the likelihood of the training data)

Maximum Likelihood (ML) estimation

- We can maximize the log of the above expression:

$$\max \log \prod_{i=1}^{m} p(y^{(i)} \mid x^{(i)}) = \sum_{i=1}^{m} \underbrace{\log\left(p(y^{(i)} \mid x^{(i)})\right)}_{-\mathcal{L}(\hat{y}^{(i)}, y^{(i)})}$$

$$= -\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Cost: $J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$
(minimize)